UNIVERSIDAD PERUANA LOS ANDES

Facultad de Ingeniería

Curso: Construcción de Software

ReservaUPLA: Sistema de Gestión de Reservas de Ambientes Universitarios

Informe técnico del caso práctico (semanas 01 a 09)

Estudiante: _____

Docente: _____

Fecha: 10 de noviembre de 2026

Huancayo, Perú

Repositorio del proyecto (GitHub): [COMPLETAR POR EL ESTUDIANTE: enlace del repositorio, por ejemplo https://github.com/usuario/ReservaUPLA]. Rama de trabajo: `feature/diseno-final`. El README explica cómo crear la base de datos por terminal, configurar los secretos y ejecutar backend, frontend y pruebas.

1. Problema, actores y alcance

Las reservas de laboratorios, auditorios y salas de la UPLA se hacían por mensajes, llamadas y papel: había duplicidad de reservas, solicitudes perdidas y ninguna trazabilidad. ReservaUPLA centraliza el flujo: el solicitante pide un ambiente, el sistema comprueba la disponibilidad, el administrador aprueba o rechaza y cada cambio queda registrado con quién lo hizo y cuándo.

- **Actores:** Administrador (decide y gestiona ambientes y usuarios), Solicitante (pide y consulta sus reservas) y Responsable de ambiente (solo consulta).
- **Alcance:** registro, validación, aprobación, consulta, disponibilidad diaria y dashboard. Fuera de alcance: correos, reservas recurrentes, calendario mensual, exportación y recuperación de contraseña.
- **Reglas del caso:** RN-01 dos reservas aprobadas del mismo ambiente no se cruzan; RN-02 todos los datos son obligatorios; RN-03 la hora de fin es posterior a la de inicio; RN-04 el horario permitido es de 07:00 a 22:00; RN-05 solo el Administrador aprueba o rechaza; RN-06 una reserva aprobada, rechazada o cancelada no cambia de estado.

2. Ciclo de vida y estándares de construcción

Se eligió un modelo **incremental e iterativo**: los requisitos del caso son estables, pero el valor se entrega por incrementos (reservar, decidir, consultar y visualizar), cada uno cerrado con su prueba. Se descartó la cascada porque no da retroalimentación temprana, y la espiral porque su gestión de riesgos es excesiva para un equipo de una persona.

2.1 Estándares

- **Nombres:** dominio en español (Reserva, Ambiente, Usuario) y términos técnicos en inglés cuando son convención (Service, Repository, Validator).
- **Carpetas:** backend en cuatro capas (Domain, Application, Infrastructure, API) y frontend por pages, components, hooks, services y utils.
- **Dependencias:** Application nunca referencia Infrastructure; Application define las interfaces e Infrastructure las implementa.
- **Commits y ramas:** «tipo: descripción» en español y una rama por tema.

Un estándar que nadie comprueba se olvida, así que los más importantes son pruebas automáticas. Por ejemplo, esta prueba falla si Application referencia Infrastructure:

Código 1. Prueba de arquitectura (`backend/ReservaUPLA.Application.Tests/ArquitecturaTests.cs`, líneas 13–18).

```
    public void Application_no_referencia_Infrastructure()
    {
```

```
        var referencias = NombresReferenciados(typeof(IComprobadorSalud).Assembly);

        Assert.DoesNotContain("ReservaUPLA.Infrastructure", referencias);
        Assert.DoesNotContain("ReservaUPLA.API", referencias);
```

3. Estructura: backend, frontend y base de datos

El backend usa ASP.NET Core (.NET 8) con Clean Architecture; el frontend, React con Vite. La base de datos PostgreSQL se crea **por terminal con scripts SQL numerados** (carpeta `database/`, del 00 al 07), sin migraciones ni tablas creadas desde el código: el esquema queda legible, revisable y reproducible desde cero.

Capa	Responsabilidad	Depende de
Domain	Entidades y reglas del negocio (Reserva, Ambiente, Usuario)	Nada
Application	Casos de uso, validadores, interfaces y mensajes	Domain
Infrastructure	EF Core, repositorios, BCrypt y JWT	Application y Domain
API	Controladores delgados, autenticación, límite de intentos y CORS	Application e Infrastructure

La API se conecta con un rol de PostgreSQL de **mínimo privilegio** (no con `postgres`), las contraseñas se guardan solo como hash BCrypt y los secretos viven en `user-secrets`, nunca en archivos versionados. El inicio de sesión permite cinco intentos por minuto y responde con el mismo mensaje si el correo no existe, está inactivo o la contraseña es incorrecta.

Figura 1. Pantalla de inicio de sesión



4. Formularios de entrada de datos

Los formularios (nueva reserva, registrar ambiente, registrar usuario e inicio de sesión) siguen las mismas reglas: etiqueta siempre visible, ayuda con ejemplo bajo el campo, error junto al campo con icono y texto (nunca solo color), foco en el primer campo con error y manejo completo con teclado. Todos se construyen con los mismos componentes reutilizables.

Figura 2. Formulario «Nueva reserva» con ambiente, horario y resumen en vivo

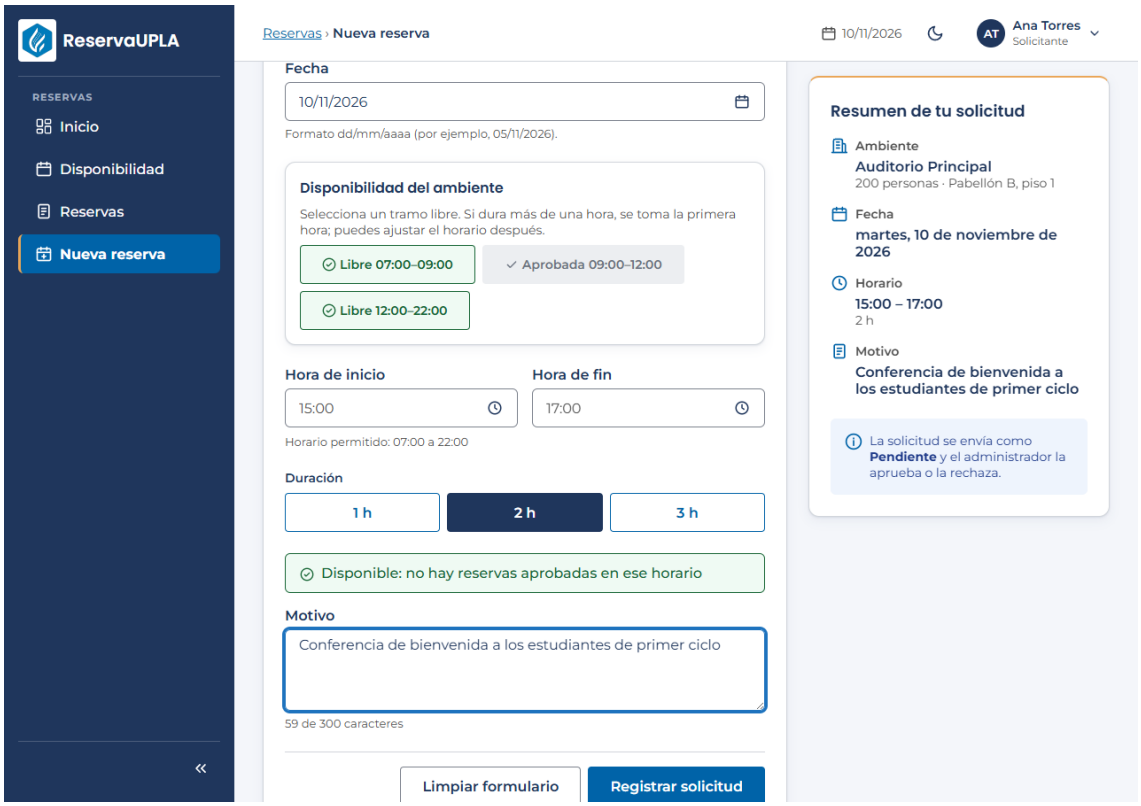
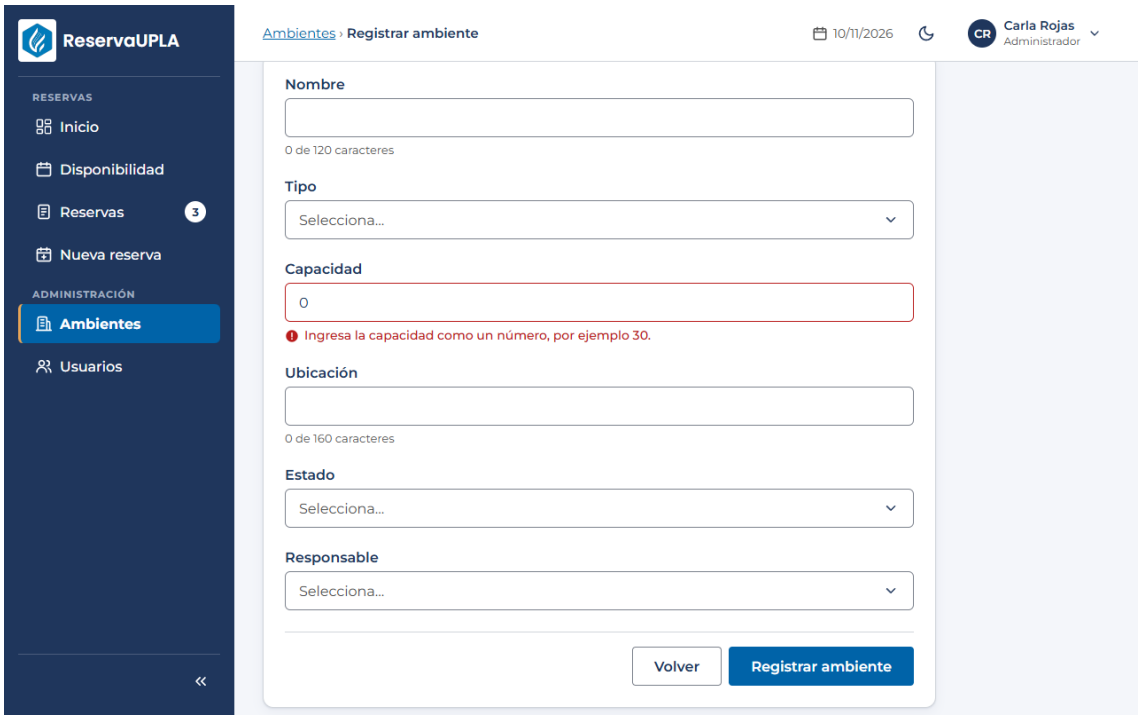


Figura 3. Formulario «Registrar ambiente» con un error de validación



5. Validaciones en frontend, backend y base de datos

Cada regla se valida en **tres capas**, con los mismos mensajes. El frontend guía al usuario mientras escribe; el backend decide, porque nunca se confía en lo que llega del cliente; y la base de datos es la última barrera. Así una regla se cumple aunque alguien se salte el formulario.

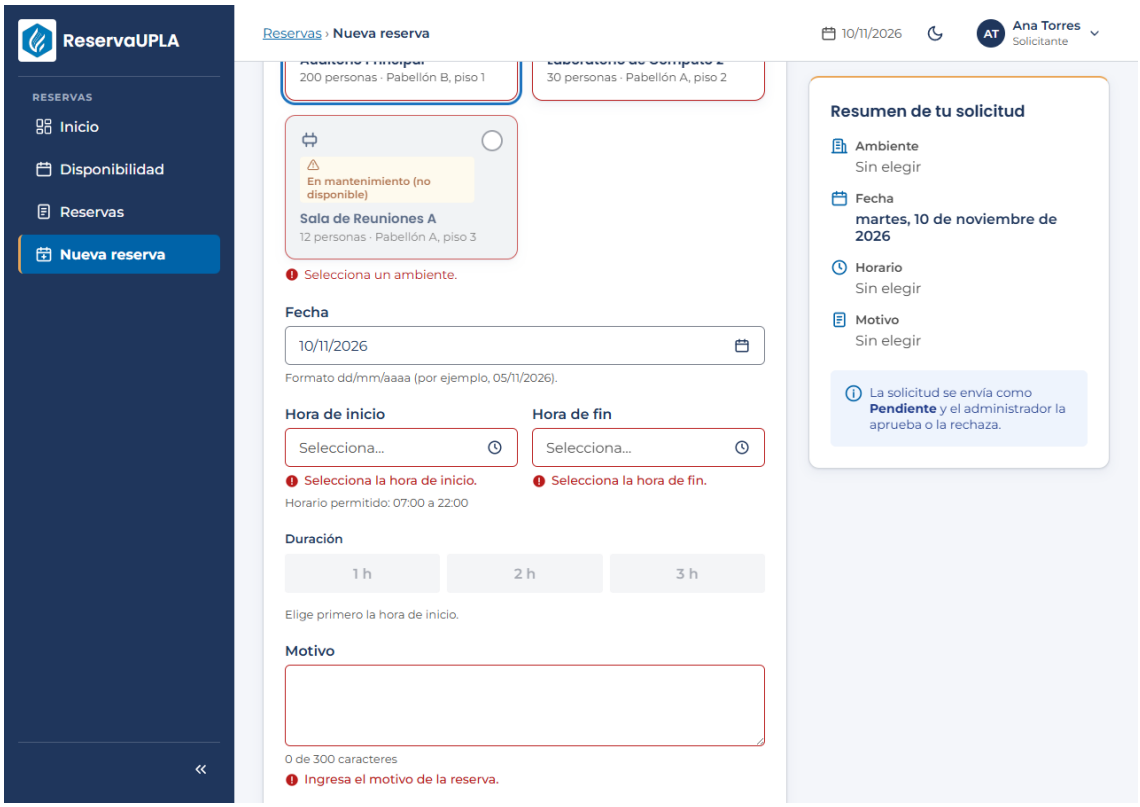
5.1 Frontend

Funciones puras, fáciles de probar, que normalizan el dato y devuelven el mensaje de error o `null`:

Código 2. Validación del correo en el frontend (`frontend/src/utils/validators.js`, líneas 37–48).

```
export function validarCorreo(
  valor,
  { mensajeVacio = MENSAJES.correoVacio, dominio = DOMINIO_INSTITUCIONAL } = {},
) {
  const correo = normalizarCorreo(valor)
  if (correo === '') return mensajeVacio
  if (correo.length > LIMITES.correo) return MENSAJES.correoDemasiadoLargo
  if (!FORMATO_CORREO.test(correo)) return MENSAJES.correoFormatoInvalido
  if (!correo.endsWith(`@${dominio}`)) return mensajeDominioNoInstitucional(dominio)
  return null
}
```

Figura 4. Enviar «Nueva reserva» vacía: un mensaje junto a cada campo y el foco en el primero



5.2 Backend

FluentValidation separa las reglas de entrada de la lógica del caso de uso. Los textos salen de una sola clase de mensajes:

Código 3. Validador de la reserva (RN-02, RN-03 y RN-04) (`backend/ReservaUPLA.Application/Validators/CrearReservaValidator.cs`, líneas 15–26).

```
    public CrearReservaValidator()
    {
        this.ReglasDeAmbiente(dto => dto.AmbienteId);
        this.ReglasDeFecha(dto => dto.Fecha);
        this.ReglasDeHoras(dto => dto.HoraInicio, dto => dto.HoraFin);

        RuleFor(x => x.Motivo)
            .Cascade(CascadeMode.Stop)
            .NotEmpty().WithMessage(MensajesValidacion.MotivoVacio)

.MaximumLength(LongitudMaximaMotivo).WithMessage(MensajesValidacion.MotivoDemasiadoLargo)
            .OverridePropertyName(CamposFormulario.Motivo);
    }
```

Las reglas que consultan datos, como el cruce de horarios (RN-01), las evalúa el servicio y responden con un conflicto explicado:

Código 4. Comprobación de cruce al aprobar (RN-01) (`backend/ReservaUPLA.Application/Services/ReservaService.cs`, líneas 200–209).

```
    private async Task<ResultadoOperacion<ReservaDetalleDto>?> ComprobarSinCruceAsync(Reserva
reserva, CancellationToken cancellationToken)
    {
        if (reserva.Estado == EstadoReserva.Pendiente
            && await _comprobador.MensajeDeConflictoAsync(reserva, cancellationToken) is { }
mensaje)
        {
            return ResultadoOperacion<ReservaDetalleDto>.Conflicto(mensaje);
        }

        return null;
    }
```

El dominio protege además las transiciones de estado (RN-06): una reserva que ya salió de «Pendiente» no se puede aprobar ni rechazar de nuevo.

Código 5. Transiciones de estado en la entidad (`backend/ReservaUPLA.Domain/Entities/Reserva.cs`, líneas 40–52).

```
    /// <summary>Pendiente → Aprobada. Solo la decide el Administrador (RN-05, lo comprueba el
servicio).</summary>
    public void Aprobar(int usuarioId, DateTimeOffset ahora)
    {
        ExigirPendiente();
        CambiarEstado(EstadoReserva.Aprobada, usuarioId, ahora);
    }

    /// <summary>Pendiente → Rechazada.</summary>
    public void Rechazar(int usuarioId, DateTimeOffset ahora)
    {
        ExigirPendiente();
        CambiarEstado(EstadoReserva.Rechazada, usuarioId, ahora);
    }
```

5.3 Base de datos

La restricción de exclusión de PostgreSQL garantiza RN-01 aunque dos personas aprueben a la vez: la segunda aprobación falla en la base de datos.

Código 6. Restricción de exclusión que impide el cruce de reservas aprobadas (`database/03_crear_tablas.sql`, líneas 56–60).

```
    -- RN-01: dos reservas Aprobadas del mismo ambiente no pueden cruzarse (rango [inicio, fin)).
    CONSTRAINT ex_reservas_aprobadas_sin_cruce EXCLUDE USING gist (
        ambiente_id WITH =,
        tsrange(fecha + hora_inicio, fecha + hora_fin) WITH &&
    ) WHERE (estado = 'Aprobada')
```

Figura 5. Formulario «Registrar usuario» con errores de validación junto a cada campo

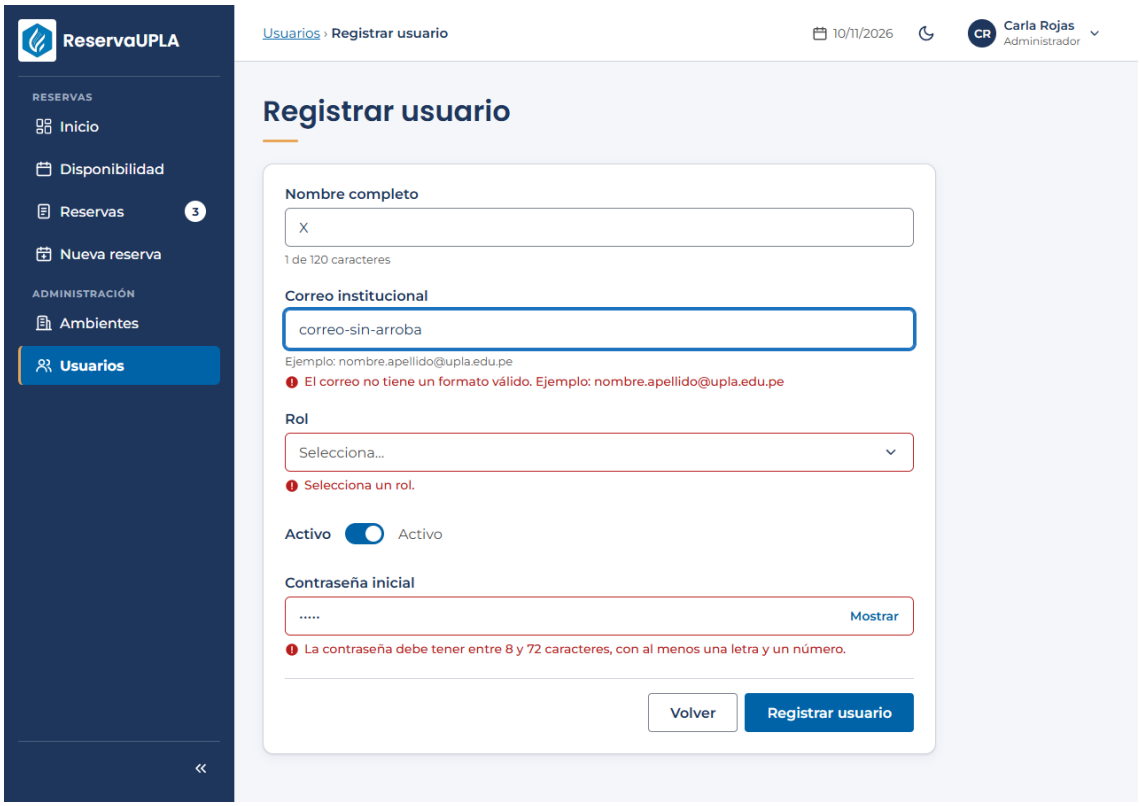


Figura 6. RN-01: aviso de disponibilidad con un tramo ocupado marcado «Choque»

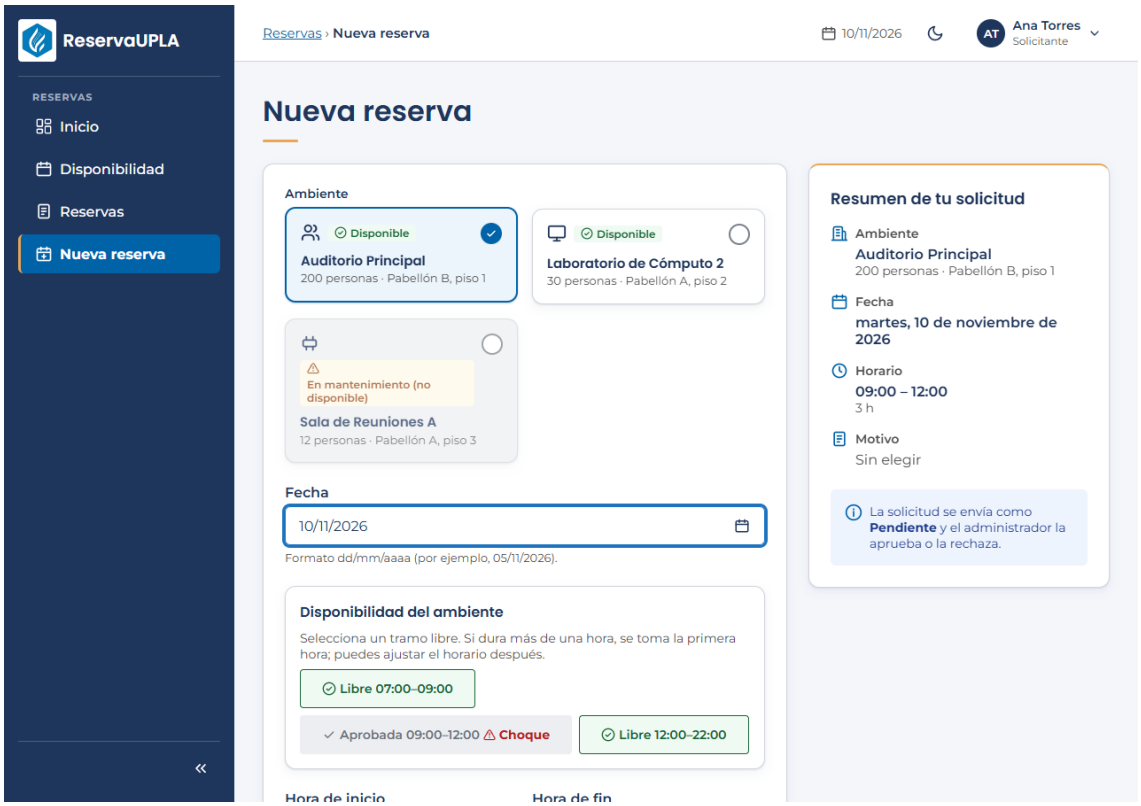
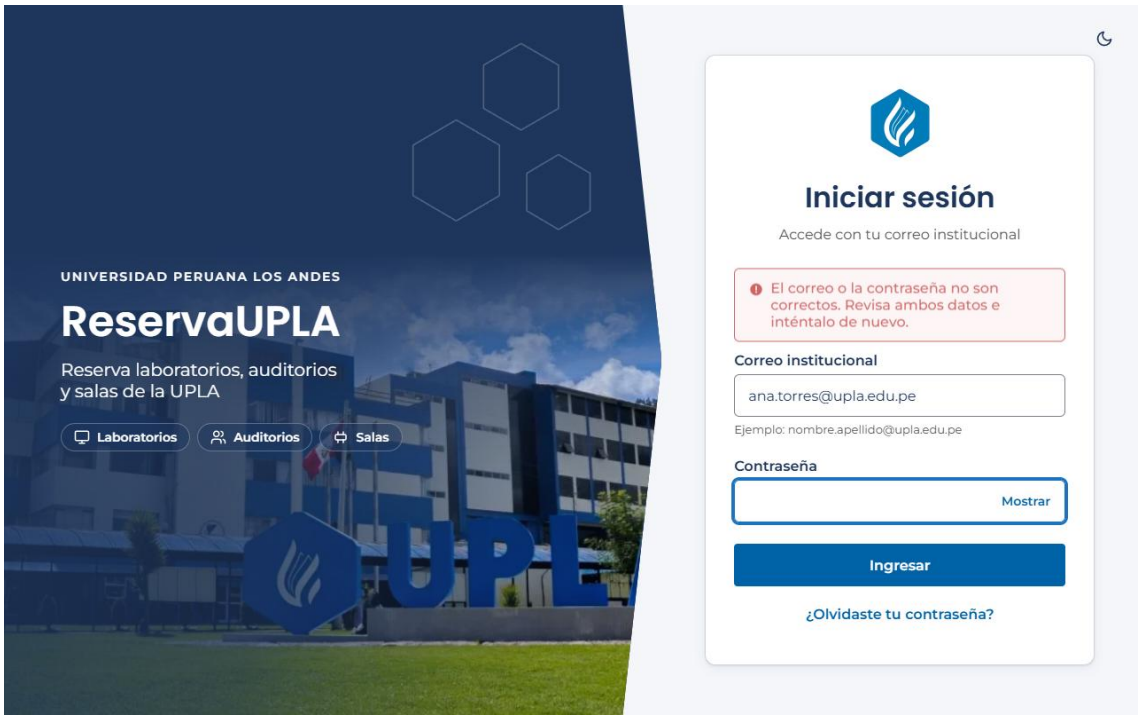


Figura 7. Credenciales incorrectas: mensaje genérico que no revela si el correo existe



6. Reutilización de componentes

Se construyeron los nueve componentes mínimos del caso (TextInput, SelectInput, DateInput, TimeInput, Button, AlertMessage, ModalConfirmacion, TableList y CardIndicador) y hooks compartidos. Cada pantalla los usa en lugar de escribir su propio botón, campo o tabla. La duplicación medida con jscpd 4.0.5, perfil de 4 líneas y 30 tokens bajó de 1,38 % a 0,93 %.

Un ejemplo: el botón único del sistema decide sus variantes, el estado de carga y el bloqueo de doble clic en un solo lugar, así que todas las pantallas que lo usan se comportan igual.

Código 7. Bloqueo de doble envío en el botón reutilizable (`frontend/src/components/ui/Button.jsx`, líneas 21–28).

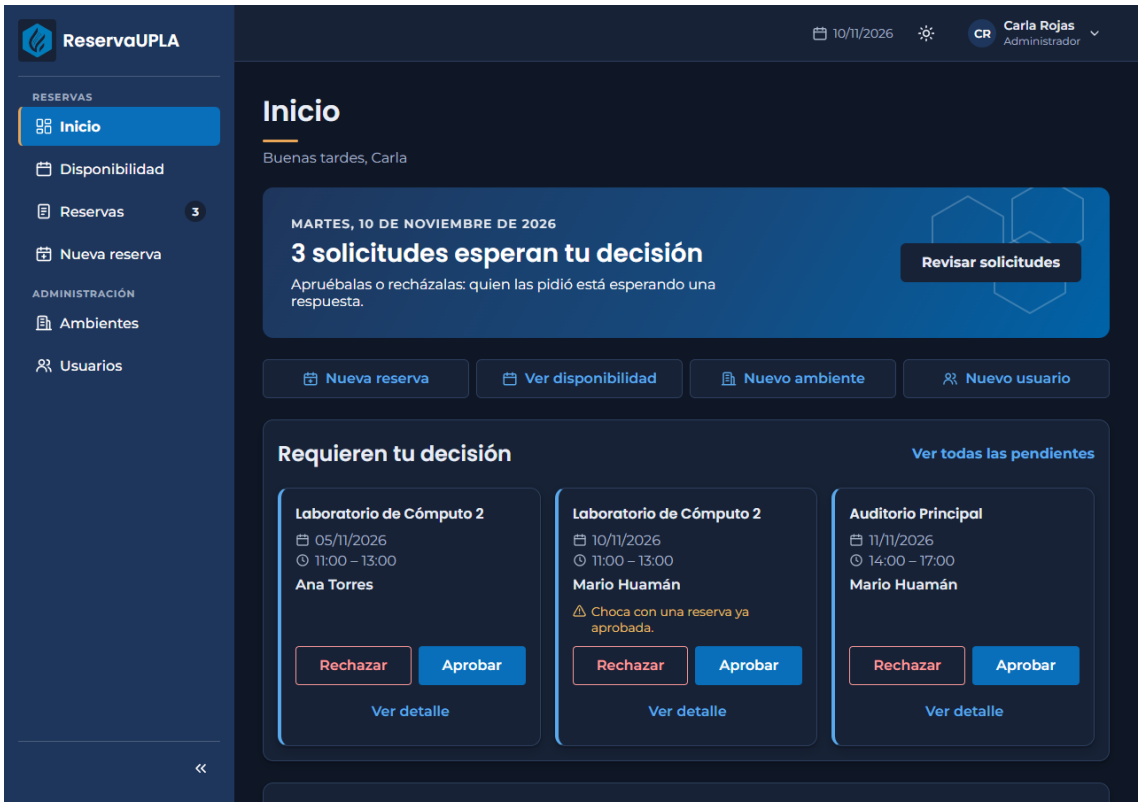
```
  const alHacerClic = (evento) => {
    if (cargando) {
      evento.preventDefault()
      return
    }
    onClick?.(evento)
  }
```

Todos los colores, radios y sombras salen de un solo archivo de tokens. Gracias a eso, el modo oscuro se agregó al final redefiniendo los tokens, sin tocar ningún componente:

Código 8. Tokens de color (tema claro) (`frontend/src/styles/tokens.css`, líneas 3–8).

```
  --color-primario: #0063a8;
  --color-marino: #1f365c;
  --color-titulo: #1e73bf;
  /* Anillo de foco: #1E73BE llega a 3:1 sobre blanco (4,9) y sobre el fondo gris (4,5). El
celeste #3696EA (2,8 sobre gris) queda solo
     decorativo. Sobre el menú marino se usa el anillo claro, porque #1E73BE solo da 2,4 ahí. */
  --color-foco: #1e73be;
```

Figura 8. El mismo Inicio del Administrador en modo oscuro



7. Refinamiento de código

El refinamiento se hizo con una red de pruebas que permite cambiar sin miedo y, sobre todo, **mirando el sistema funcionando con datos reales**. Eso reveló defectos que ninguna prueba cubría, y cada uno quedó corregido con su prueba:

- «Tu próxima reserva» mostraba una reserva cancelada: ahora prefiere la aprobada y luego la pendiente.
- Las listas del Responsable incluían reservas vencidas: se filtran las vigentes.
- El menú lateral contraído quedaba de 80 píxeles de alto por una regla de CSS: se corrigió y se añadió una prueba.
- Textos recortados en la línea de tiempo de disponibilidad: el texto visible depende del ancho del tramo.

8. Salida de datos y dashboard

El Administrador ve lo que debe decidir, la agenda de hoy y los indicadores; el Solicitante, su próxima reserva y el estado de sus solicitudes; y el Responsable solo consulta, sin acciones de decisión (RN-05).

Figura 9. Inicio del Administrador: decisiones pendientes, agenda de hoy e indicadores

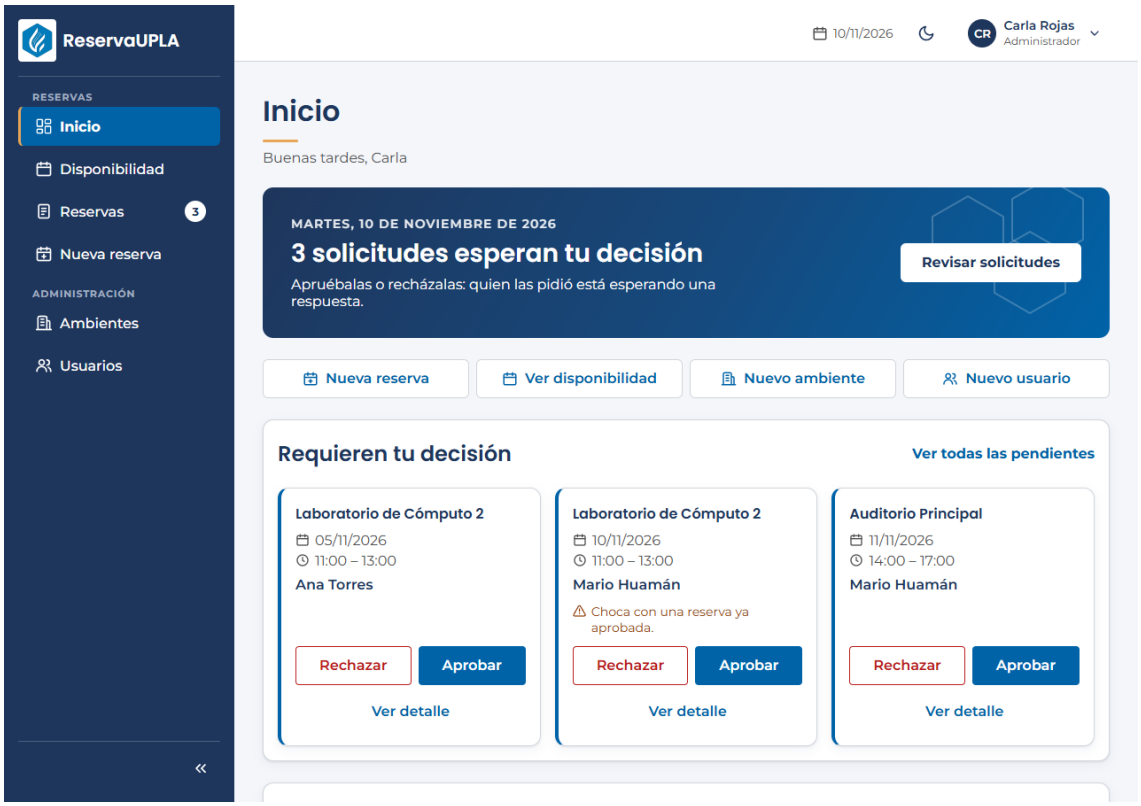


Figura 10. Indicadores por estado, ambiente más utilizado e indicadores de gestión

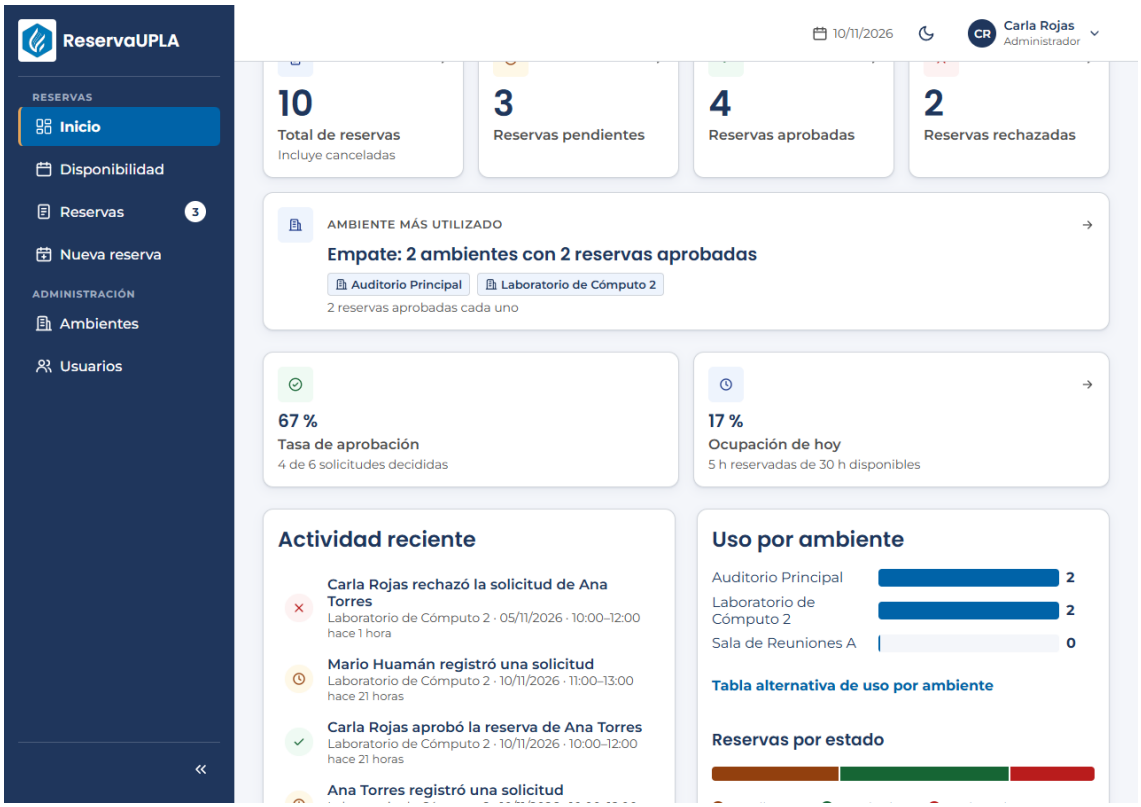


Figura 11. Listado de reservas con pestañas por estado, filtros y tabla

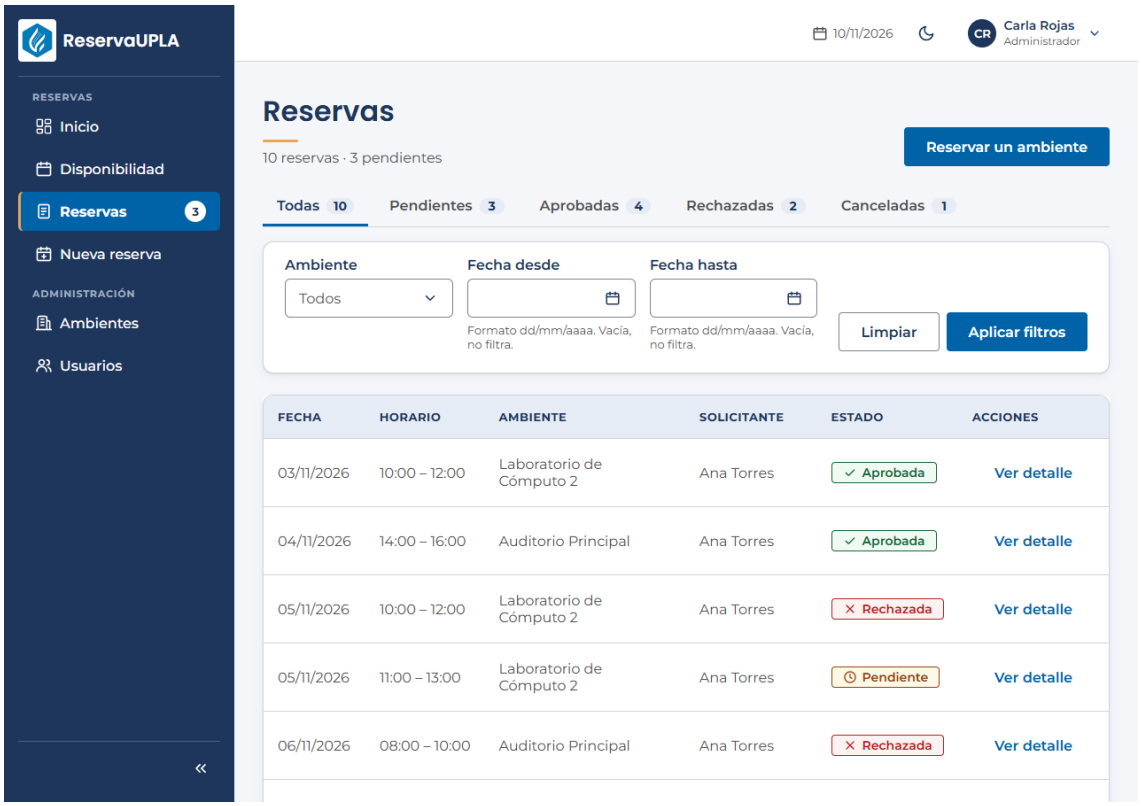


Figura 12. Reserva pendiente que choca con una aprobada: «Aprobar» queda deshabilitado y se explica por qué

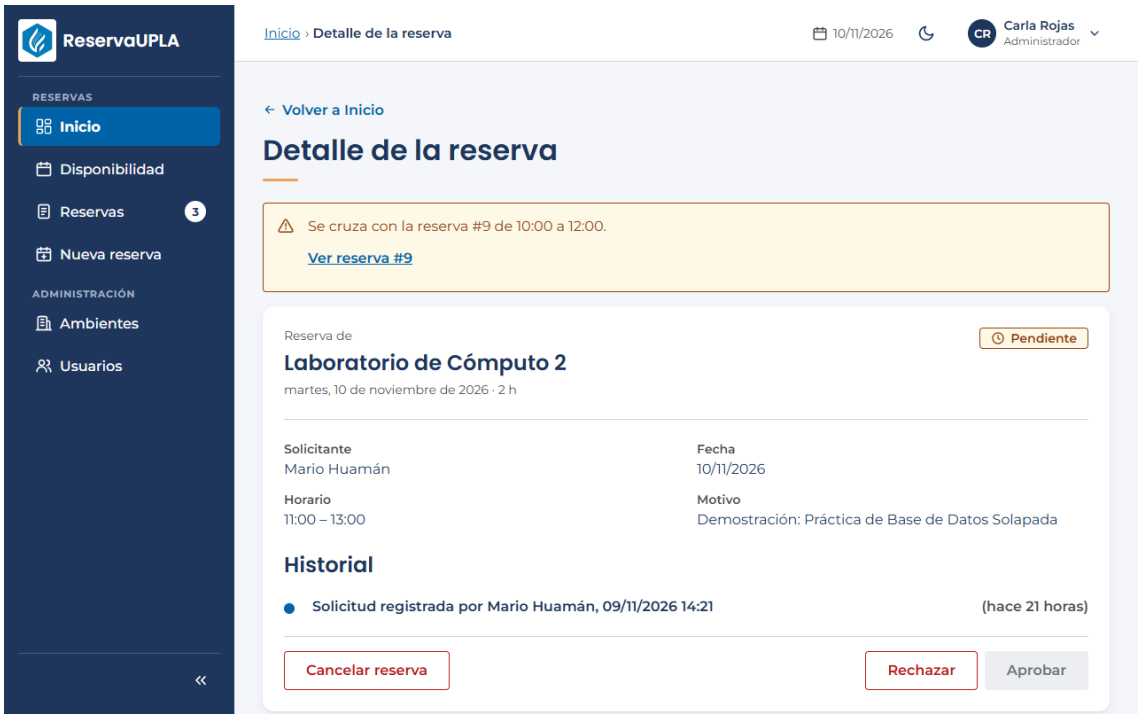


Figura 13. Confirmación antes de decidir: el modal dice lo que se va a hacer

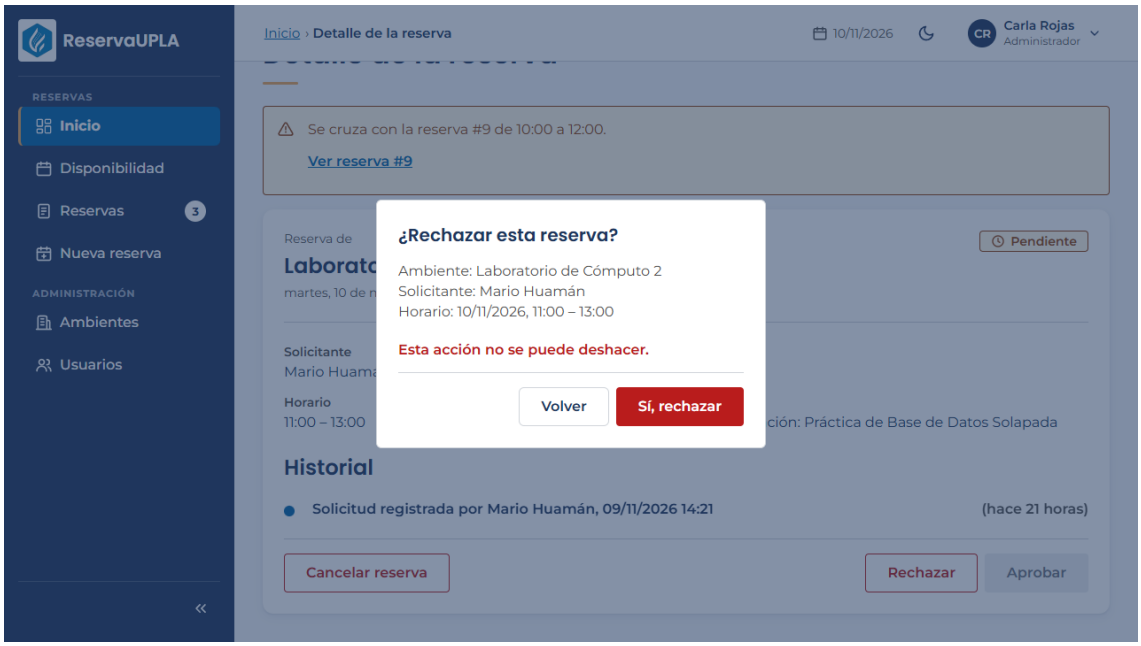
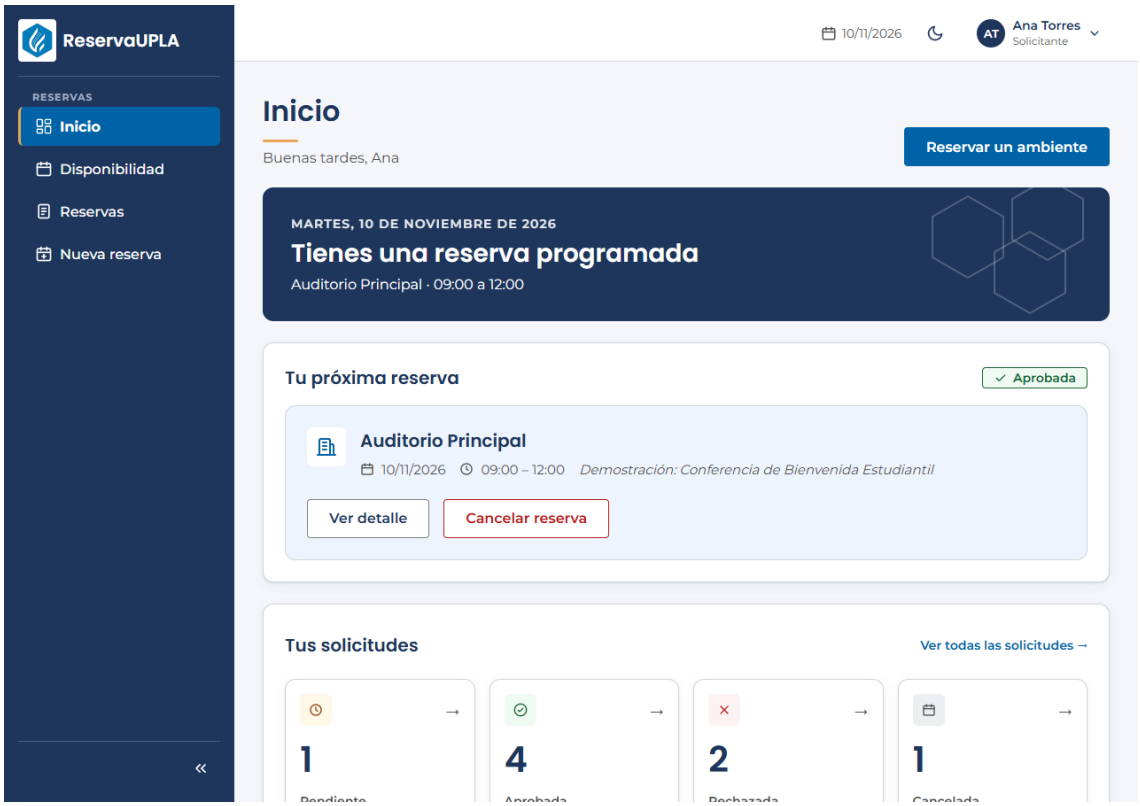


Figura 14. Vista diaria de disponibilidad por ambiente



Figura 15. Inicio del Solicitante



9. Calidad, pruebas y límites

Medida	Resultado	Fecha y entorno
Pruebas del frontend	1375 en 62 archivos	2026-11-10, npm --prefix frontend test
Pruebas de Application	162	2026-11-10, WSL (Ubuntu), sin base de datos
Pruebas de Infrastructure	14	2026-11-10, WSL con PostgreSQL 18 real
Pruebas de la API	60 de 60	2026-11-07, Windows, con PostgreSQL real (60 de 60)
Peso del build (gzip)	JS 130,02 kB, CSS 15,79 kB	2026-11-10

También se verificó accesibilidad (axe, contraste AA, objetivos táctiles de al menos 44 píxeles, foco visible y teclado) y el uso en pantallas de 375 píxeles sin desplazamiento horizontal.

Límites declarados: la última ejecución completa de las pruebas de la API, del extremo a extremo y de la accesibilidad automatizada es del 7 de noviembre (desde entonces Windows bloquea ejecutar el backend compilado localmente y se probó en WSL); no se midió el rendimiento con carga.

10. Reflexión final

«¿Cómo contribuye la aplicación de estándares, la validación de datos, la reutilización de componentes y el refinamiento de código a mejorar la calidad, la mantenibilidad y la evolución del sistema?»

Antes pensaba que calidad era que «funcione»; aquí aprendí que es poder **cambiar sin miedo**. Los **estándares** dieron orden y, convertidos en pruebas automáticas, se respetan solos. La **validación en tres capas** da confianza: el frontend ayuda, el backend decide y la base de datos garantiza, como se vio con la restricción que impide cruzar reservas. La **reutilización** hace que cada cambio se haga una sola vez, como el modo oscuro. El **refinamiento** con pruebas y con datos reales encuentra lo que las pruebas no ven.

- **Calidad:** las reglas se cumplen aunque alguien se salte el formulario, y los mensajes son iguales en las tres capas.
- **Mantenibilidad:** capas con dependencias en una dirección, una sola fuente de textos y colores, y una red de pruebas que avisa si algo se rompe.
- **Evolución:** la vista de disponibilidad, los indicadores de gestión, la actividad reciente y el modo oscuro se añadieron sin reescribir lo anterior.

Pendiente: **[COMPLETAR POR EL ESTUDIANTE: resultados de la prueba de usabilidad con compañeros]**.